OPERATING SYSTEMS

UE25CS242A

PES UNIVERSITY

CELEBRATING 50 YEARS

Memory Management

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS

Slides Credits for all the PPTs of this course



- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

Page replacement, Allocation of frames, Thrashing

Pavan A C
Department of Computer Science & Engineering

OPERATING SYSTEMS

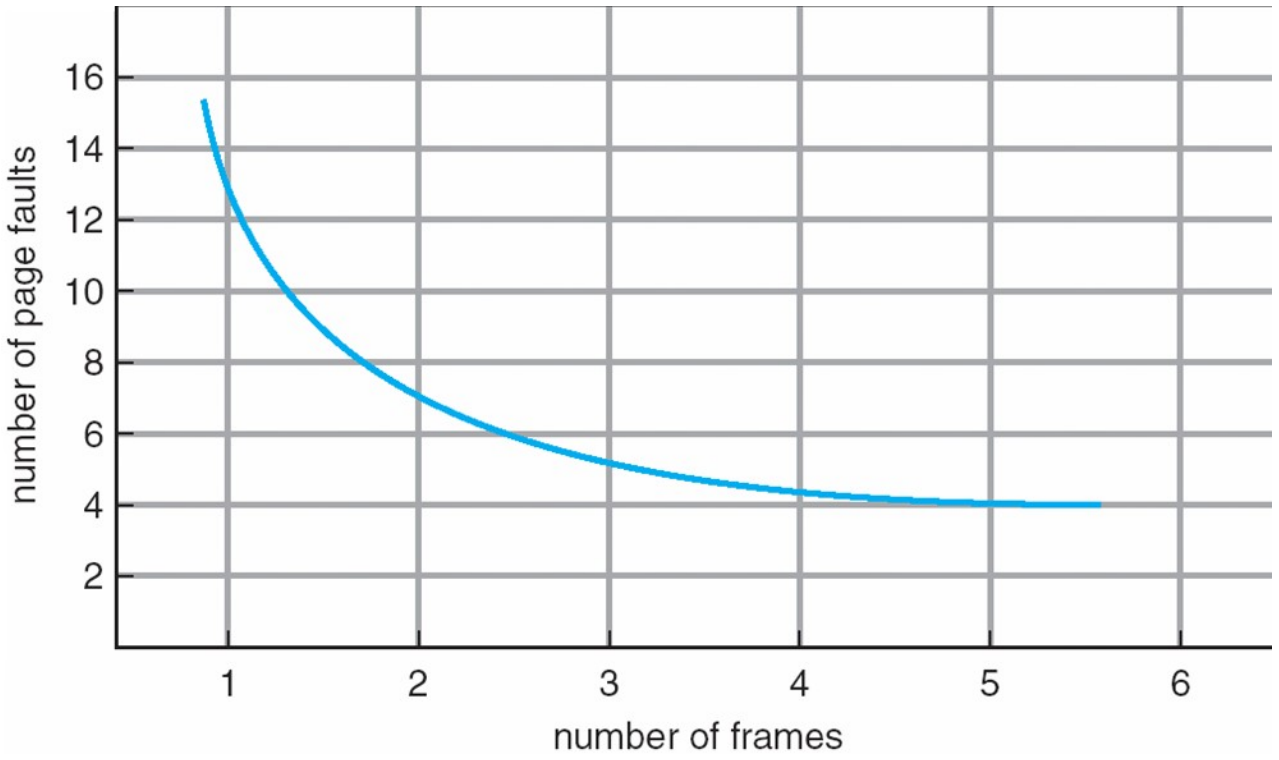
Page and Frame Replacement Algorithms



- **Frame-allocation algorithm** determines
  - How many frames to give each process
  - Which frames to replace
- **Page-replacement algorithm**
  - Want lowest page-fault rate on both first access and re-access
- Evaluate algorithm by running it on a particular string of memory references (reference string) and computing the number of page faults on that string
  - String is just page numbers, not full addresses
  - Repeated access to the same page does not cause a page fault
  - Results depend on number of frames available
- In all our examples, the **reference string** of referenced page numbers is

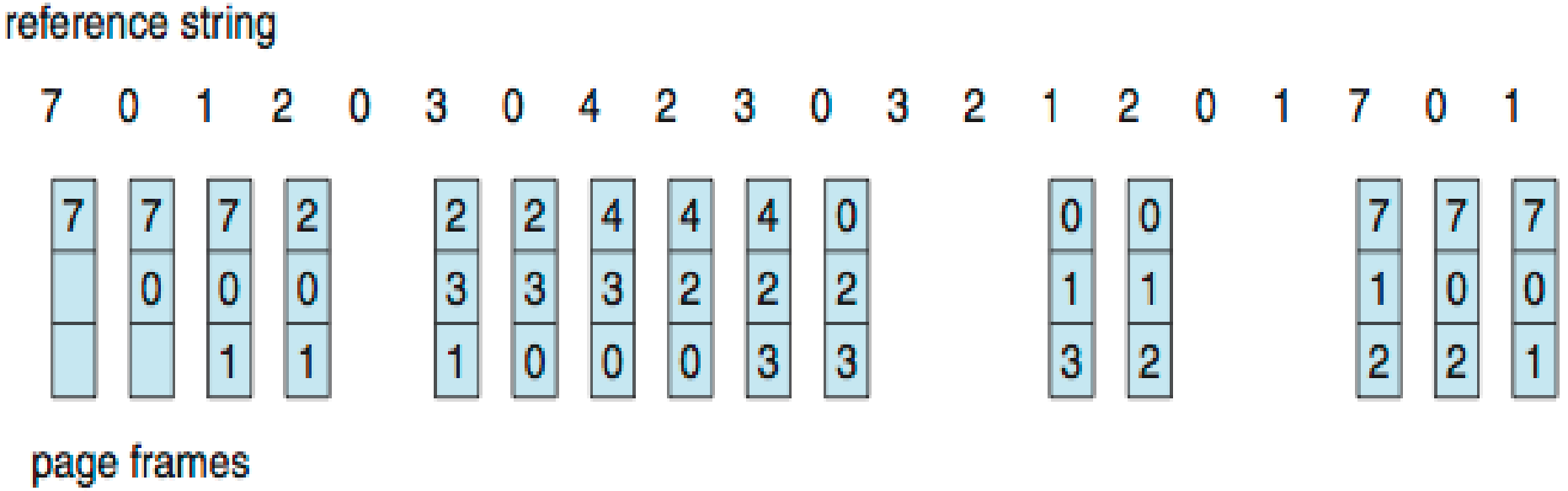
  7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1

Graph of Page Faults Versus The Number of Frames



First-In-First-Out (FIFO) Algorithm

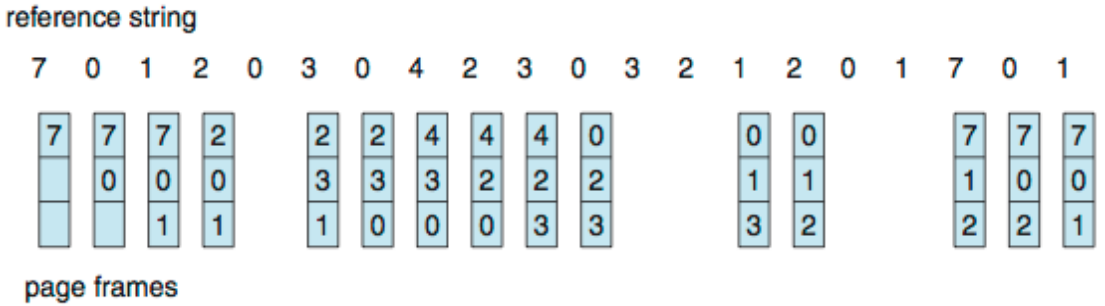
- Reference string: **7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1**
- 3 frames (3 pages can be in memory at a time per process)



OPERATING SYSTEMS

First-In-First-Out (FIFO) Algorithm

- Reference string: **7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1**
- 3 frames (3 pages can be in memory at a time per process)

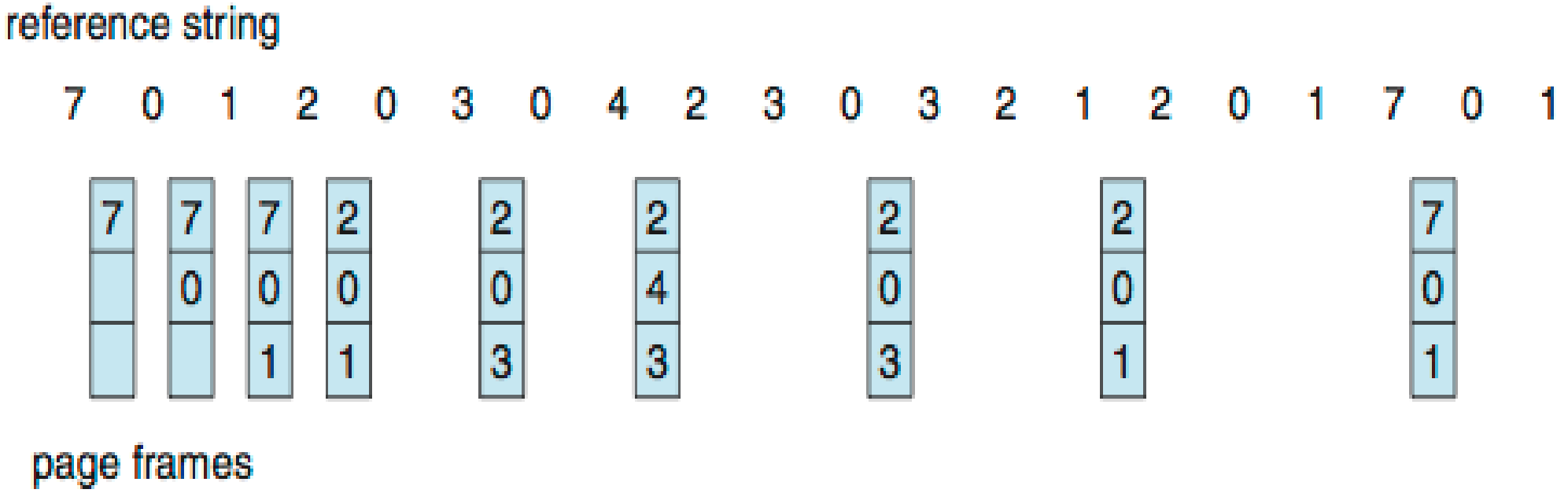


- Can vary by reference string: consider 1,2,3,4,1,2,5,1,2,3,4,5
  - Adding more frames can cause more page faults!
    - 4 **Belady's Anomaly**
- How to track ages of pages?
  - Just use a FIFO queue

OPERATING SYSTEMS

Optimal Algorithm – Theoretical Best

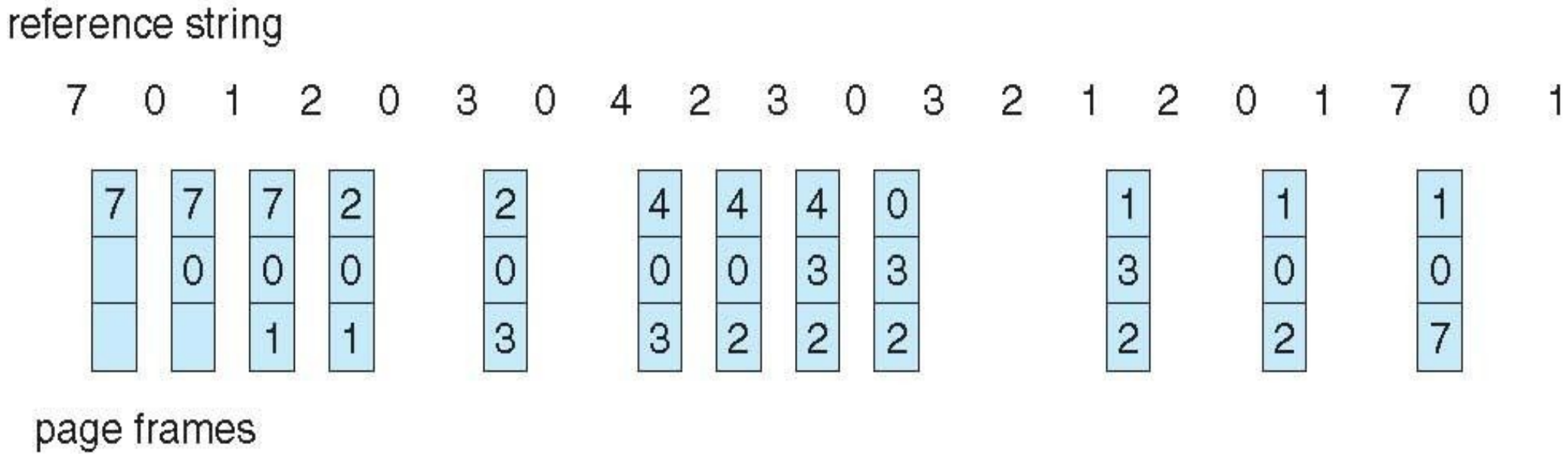
- Replace page that will not be used for longest period of time
  - 9 is optimal for the example



OPERATING SYSTEMS

Least Recently Used (LRU) Algorithm

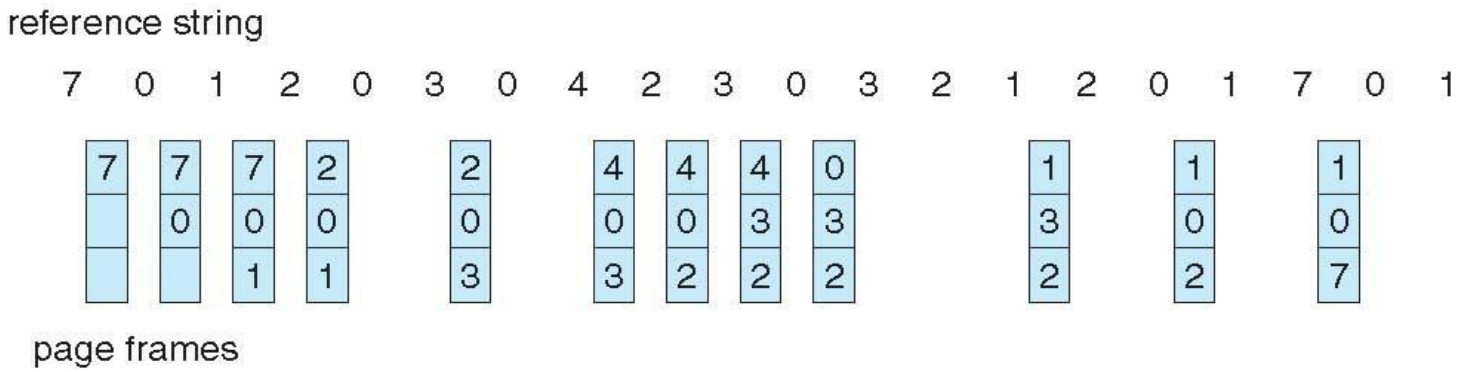
- Use past knowledge rather than future
- Replace page that has not been used in the most amount of time
- Associate time of last use with each page



OPERATING SYSTEMS

Least Recently Used (LRU) Algorithm

- Use past knowledge rather than future
- Replace page that has not been used in the most amount of time
- Associate time of last use with each page



- 12 faults – better than FIFO but worse than OPT
- Generally good algorithm and frequently used
- But how to implement?

OPERATING SYSTEMS

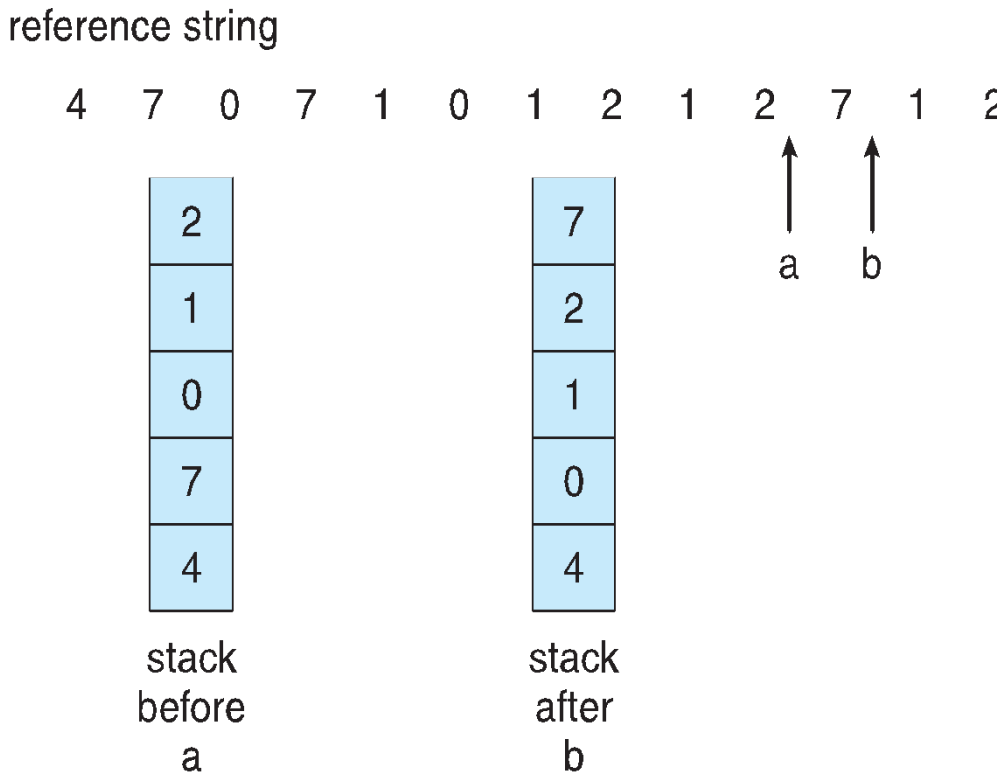
LRU Algorithm (Cont.)

- Counter implementation
  - Every page entry has a counter; every time page is referenced through this entry, copy the clock into the counter
  - When a page needs to be changed, look at the counters to find smallest value
    - Search through table needed
- Stack implementation
  - Keep a stack of page numbers in a double link form:
  - Page referenced:
    - move it to the top
    - requires 6 pointers to be changed
  - But each update more expensive
  - No search for replacement - bottom of the stack is our victim
- LRU and OPT are cases of **stack algorithms** that don't have Belady's Anomaly

OPERATING SYSTEMS

Use Of A Stack to Record Most Recent Page References

- LRU and OPT are cases of **stack algorithms** that don't have Belady's Anomaly
- Use Of A Stack to Record Most Recent Page References



OPERATING SYSTEMS

Allocation of Frames



- How do we allocate the fixed amount of free memory among the various processes?

Example:

- Consider a single-user system with 128 KB of memory composed of pages 1 KB in size. Therefore, this system has 128 frames.
- The operating system may take 35 KB, leaving 93 frames for the user process.
- When a user process started execution, under pure demand paging, it would generate a sequence of page faults.
- The first 93 page faults would all get free frames from the free-frame list.
- When the free-frame list is exhausted, a page-replacement algorithm would be used.
- When the process is terminated, the 93 frames would once again be placed on the free-frame list

OPERATING SYSTEMS

Allocation of Frames



Minimum number of frames:

- One reason for allocating at least a minimum number of frames involves performance.
- As the number of frames allocated to each process decreases, the page-fault rate increases, slowing process execution
- When a page fault occurs before an executing instruction is complete, the instruction must be restarted. Consequently, we must have enough frames to hold all the different pages that any single instruction can reference
- The minimum number of frames is defined by the computer architecture.
- The maximum number is defined by the amount of available physical memory.
- In between, we are still left with significant choice in frame allocation

OPERATING SYSTEMS

Frame Allocation Algorithms



Equal allocation:

- The easiest way to split m frames among n processes is to give everyone an equal share, **m/n** frames (ignoring frames needed by the operating system)

  Example:

  - If there are 93 frames and 5 processes, each process will get 18 frames.
  - The three leftover frames can be used as a free-frame buffer pool.

Frame Allocation Algorithms



Proportional allocation:

- Allocate available memory to each process according to its size
- Let the size of the virtual memory for process p_i be s_i and define $S = \sum s_i$
- If the total number of available frames is m, allocate a_i frames to process p_i , where a_i is approximately $a_i = s_i / S \times m$
- Adjust each a_i to be an integer that is greater than the minimum number of frames required by the instruction set, with a sum not exceeding m
- **Example**: To allocate 62 frames between two processes, one of 10 pages and one of 127 pages
  - $10/137 \times 62 \approx 4$
  - $127/137 \times 62 \approx 57$

OPERATING SYSTEMS

Global versus Local Allocation

- Page-replacement algorithms can be classified into two broad categories: global replacement and local replacement
- **Global replacement** allows a process to select a replacement frame from the set of all frames, even if that frame is currently allocated to some other process; that is, one process can take a frame from another.
- **Local replacement** requires that each process select from only its own set of allocated frames.
- **Example**: Consider a global replacement scheme wherein high-priority processes are allowed to select frames from low-priority processes for replacement.
- A process can select a replacement from among its own frames or the frames of any lower-priority process. This approach allows a high-priority process to increase its frame allocation at the expense of a low-priority process
- With a local replacement strategy, the number of frames allocated to a process does not change.
- Global replacement generally results in greater system throughput and is therefore the more commonly used method.

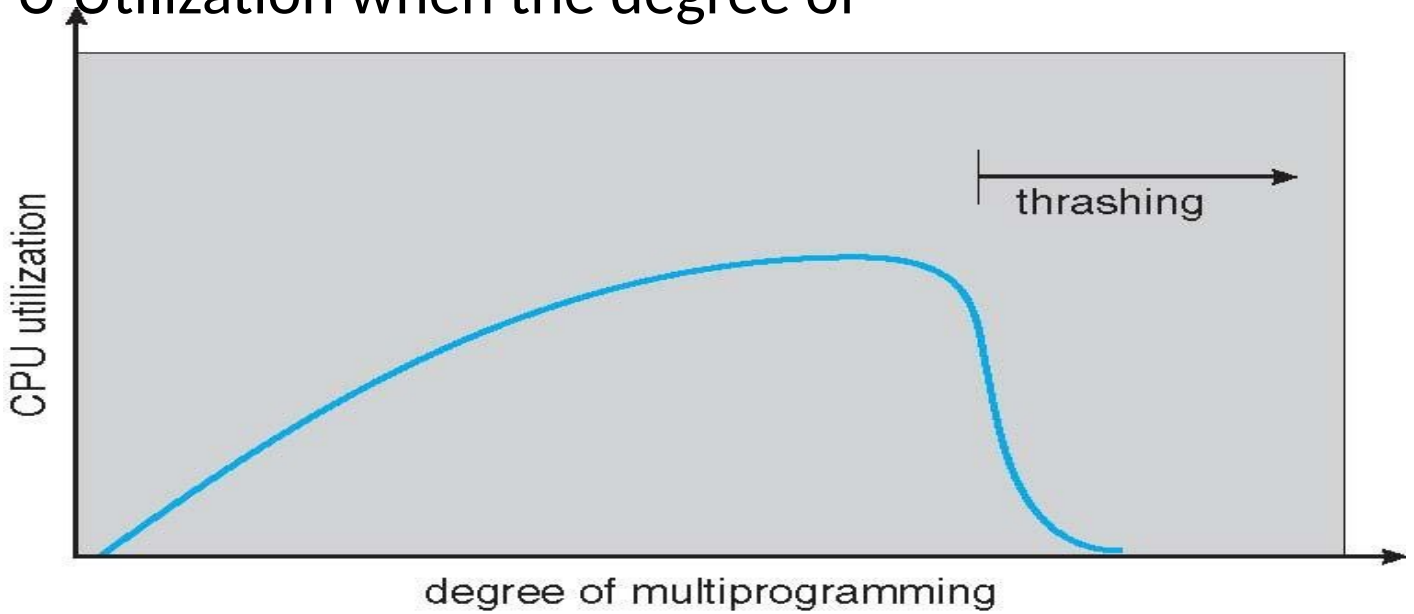
Thrashing



- If a process does not have “enough” pages, the page-fault rate is very high
  - Page fault to get page
  - Replace existing frame
  - But quickly need replaced frame back
  - This leads to:
    - Low CPU utilization
    - Operating system thinking that it needs to increase the degree of multiprogramming
    - Another process added to the system
- **Thrashing** ≡ a process is busy swapping pages in and out

Cause of Thrashing

- Thrashing results in severe performance problems.
- Consider the scenario of the paging systems performance.
- As the degree of multiprogramming increases, CPU utilization also increases.
- Why there is a decrease in the CPU Utilization when the degree of multiprogramming is increased?.
- How this will be handled?





THANK YOU

Pavan A C

Department of Computer Science & Engineering